Processingをはじめよう

第11章

配列

目次

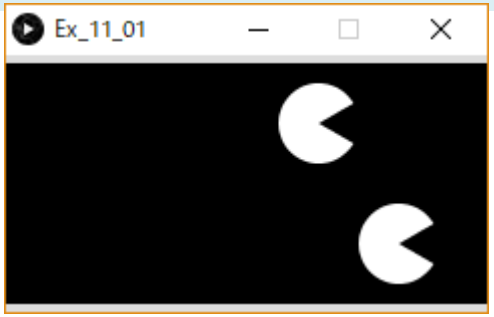
- 変数から配列へ
- 配列の作り方
- 配列と繰り返し
- オブジェクトの配列
- Robot 9: Arrays

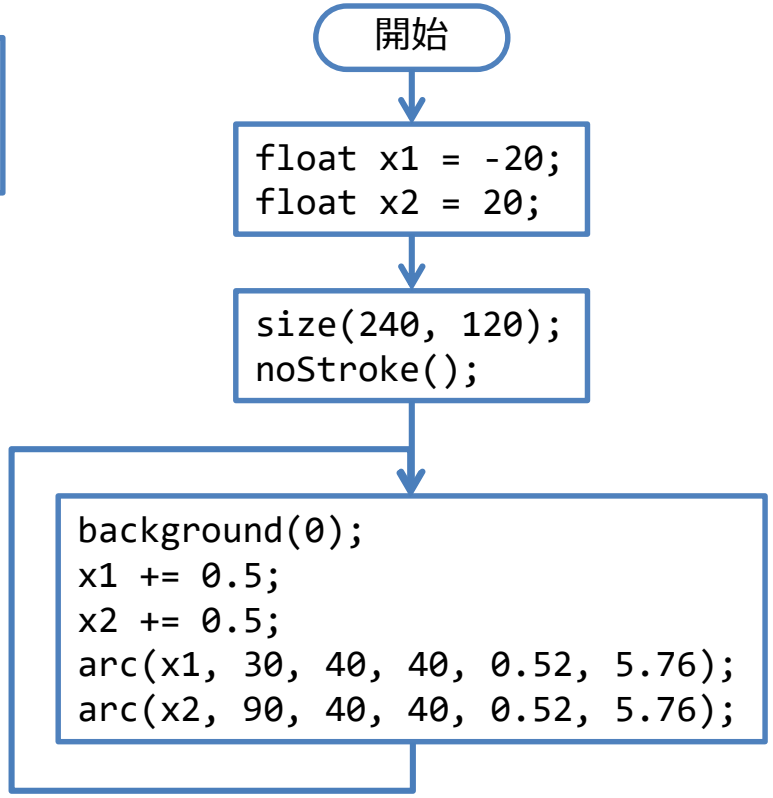
変数から配列へ

- Example 11-1 変数がたくさん

```
float x1 = -20;
float x2 = 20;

void setup() {
  size(240, 120);
  noStroke();
}

void draw() {
  background(0);
  x1 += 0.5;
  x2 += 0.5;
  arc(x1, 30, 40, 40, 0.52, 5.76);
  arc(x2, 90, 40, 40, 0.52, 5.76);
}
```

Example 8-3 の図形を
2つ表示するために
変数を2つ使う





変数から配列へ

- Example 11-2 変数が多すぎる

```
float x1 = -10;
float x2 = 10;
float x3 = 35;
float x4 = 18;
float x5 = 30;

void setup() {
  size(240, 120);
  noStroke();
}
```

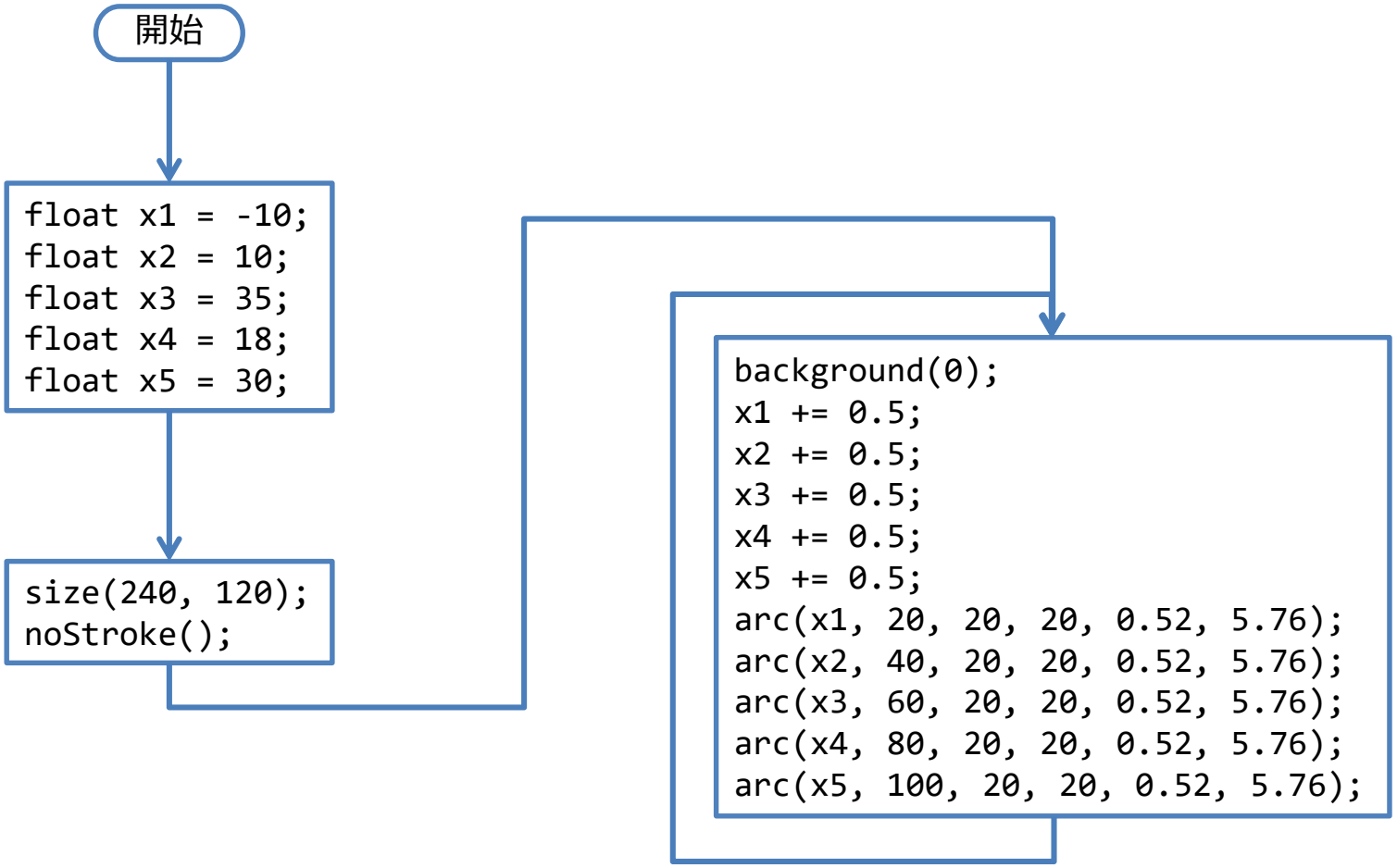
```
void draw() {
  background(0);
  x1 += 0.5;
  x2 += 0.5;
  x3 += 0.5;
  x4 += 0.5;
  x5 += 0.5;
  arc(x1, 20, 20, 20, 0.52, 5.76);
  arc(x2, 40, 20, 20, 0.52, 5.76);
  arc(x3, 60, 20, 20, 0.52, 5.76);
  arc(x4, 80, 20, 20, 0.52, 5.76);
  arc(x5, 100, 20, 20, 0.52, 5.76);
}
```

Example 8-3 の図形を
５つ表示するために
変数を５つ使う

変数が多くなると
同じような処理が
多くなってくる

図形を3000個
表示するには？

変数から配列へ

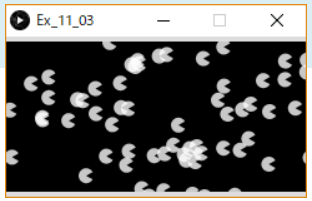

開始
float x1 = -10;
float x2 = 10;
float x3 = 35;
float x4 = 18;
float x5 = 30;
size(240, 120);
noStroke();
background(0);
x1 += 0.5;
x2 += 0.5;
x3 += 0.5;
x4 += 0.5;
x5 += 0.5;
arc(x1, 20, 20, 20, 0.52, 5.76);
arc(x2, 40, 20, 20, 0.52, 5.76);
arc(x3, 60, 20, 20, 0.52, 5.76);
arc(x4, 80, 20, 20, 0.52, 5.76);
arc(x5, 100, 20, 20, 0.52, 5.76);

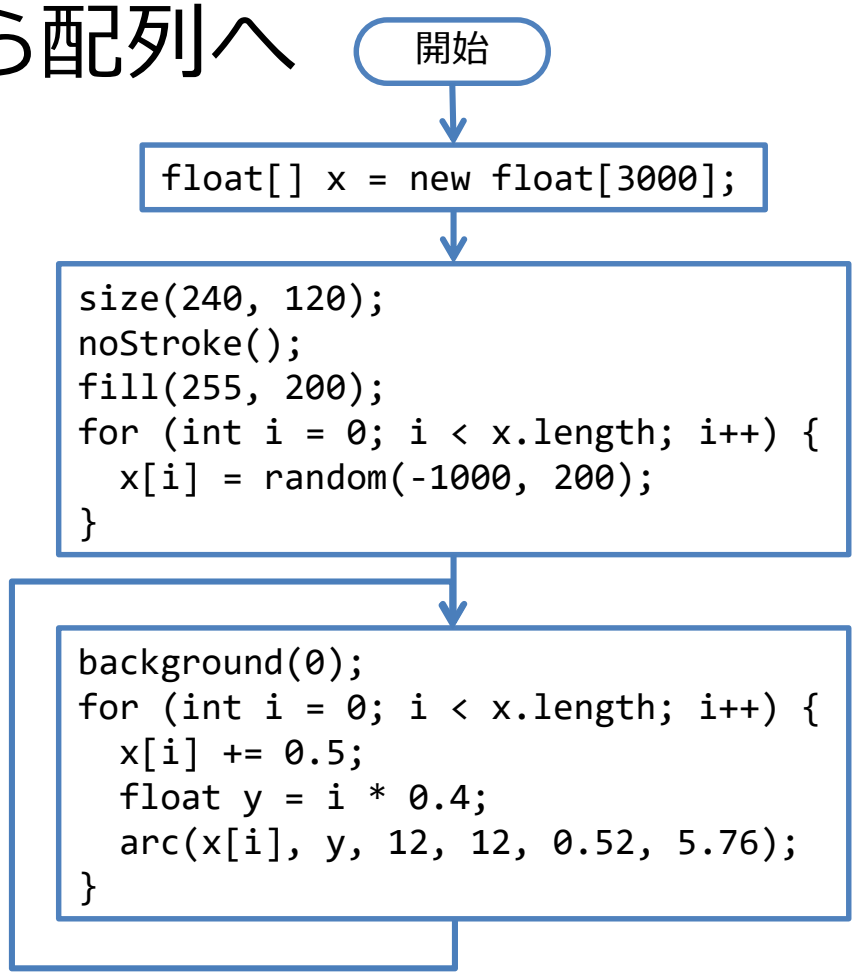
変数から配列へ

- Example 11-3 変数から配列へ

```
float[] x = new float[3000];

void setup() {
  size(240, 120);
  noStroke();
  fill(255, 200);
  for (int i = 0; i < x.length; i++) {
    x[i] = random(-1000, 200);
  }
}

void draw() {
  background(0);
  for (int i = 0; i < x.length; i++) {
    x[i] += 0.5;
    float y = i * 0.4;
    arc(x[i], y, 12, 12, 0.52, 5.76);
  }
}
```

Example 8-3 の図形を3000個表示するために配列を使う





配列の作り方

1. データ型を定義し、配列を宣言

データ型[] 配列名;

```
float[] x;
```

2. new演算子を使って配列を生成し、長さ（要素数）を定義

配列名 = new データ型[要素数];

```
x = new float[3000];
```

配列xは0番目から
2999番目までの要素がある

3. 各要素に値を代入

配列の長さ x.length は3000

```
for (int i = 0; i < x.length; i++) {
  x[i] = random(-1000, 200);
}
```

配列の作り方

- Example 11-4 配列の宣言と代入

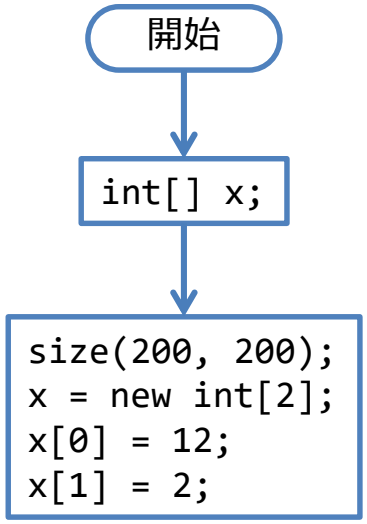
```
int[] x;  //配列の宣言

void setup() {
  size(200, 200);
  x = new int[2];  // 配列の生成
  x[0] = 12;       // x[0]に値を代入
  x[1] = 2;        // x[1]に値を代入
}
```

配列の宣言
　型[] 配列名;

配列の生成
　配列名 = new 型[要素数];

配列の各要素に値を代入

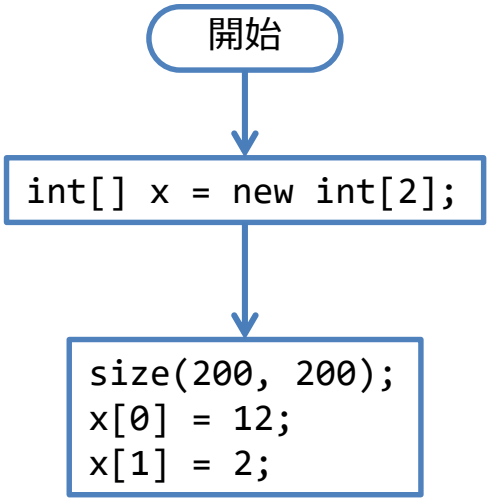
　配列名[0] = 値;
　配列名[1] = 値;
　　:



配列の作り方

- Example 11-5 コンパクトな作り方

```
int[] x = new int[2];  // 配列の宣言と生成

void setup() {
  size(200, 200);
  x[0] = 12;           // x[0]に値を代入
  x[1] = 2;            // x[1]に値を代入
}
```



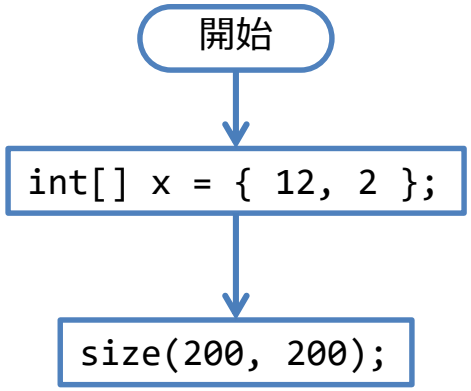
配列の宣言と生成を同時に行う

型[] 配列名 = new 型[要素数];

配列の作り方

- Example 11-6 一息に作る方法

```
int[] x = { 12, 2 }; //配列の宣言、生成、代入

void setup() {
  size(200, 200);
}
```



配列の宣言、生成、代入を同時に行う

型[] 配列名 = {値1, 値2, …};

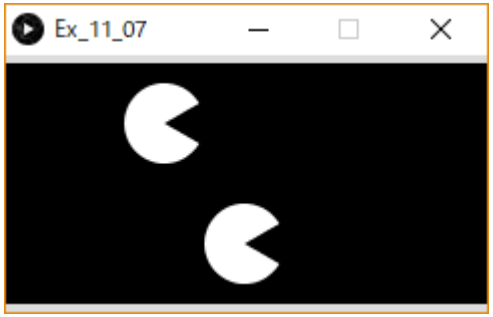
代入する値の数で要素数が決まる

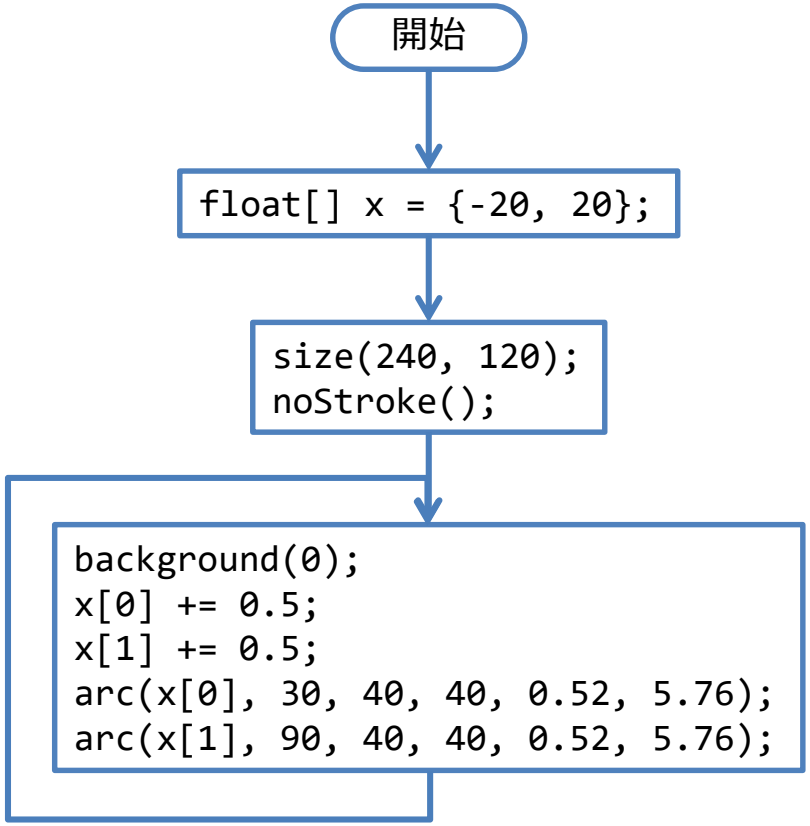
配列の作り方

- Example 11-7 配列のおさらい

```
float[] x = {-20, 20};

void setup() {
  size(240, 120);
  noStroke();
}

void draw() {
  background(0);
  x[0] += 0.5;
  x[1] += 0.5;
  arc(x[0], 30, 40, 40, 0.52, 5.76);
  arc(x[1], 90, 40, 40, 0.52, 5.76);
}
```

Example 11-1を
配列を使って
書き換える





配列と繰り返し

- 配列とforループを使って、大きな配列の処理が容易になる
- 例
  - 配列のi番目の要素に初期値を代入する
  - forループ内で配列を使った処理を要素を更新しながら繰り返す など

配列と繰り返し

- Example 11-8 forループで配列に値を満たす

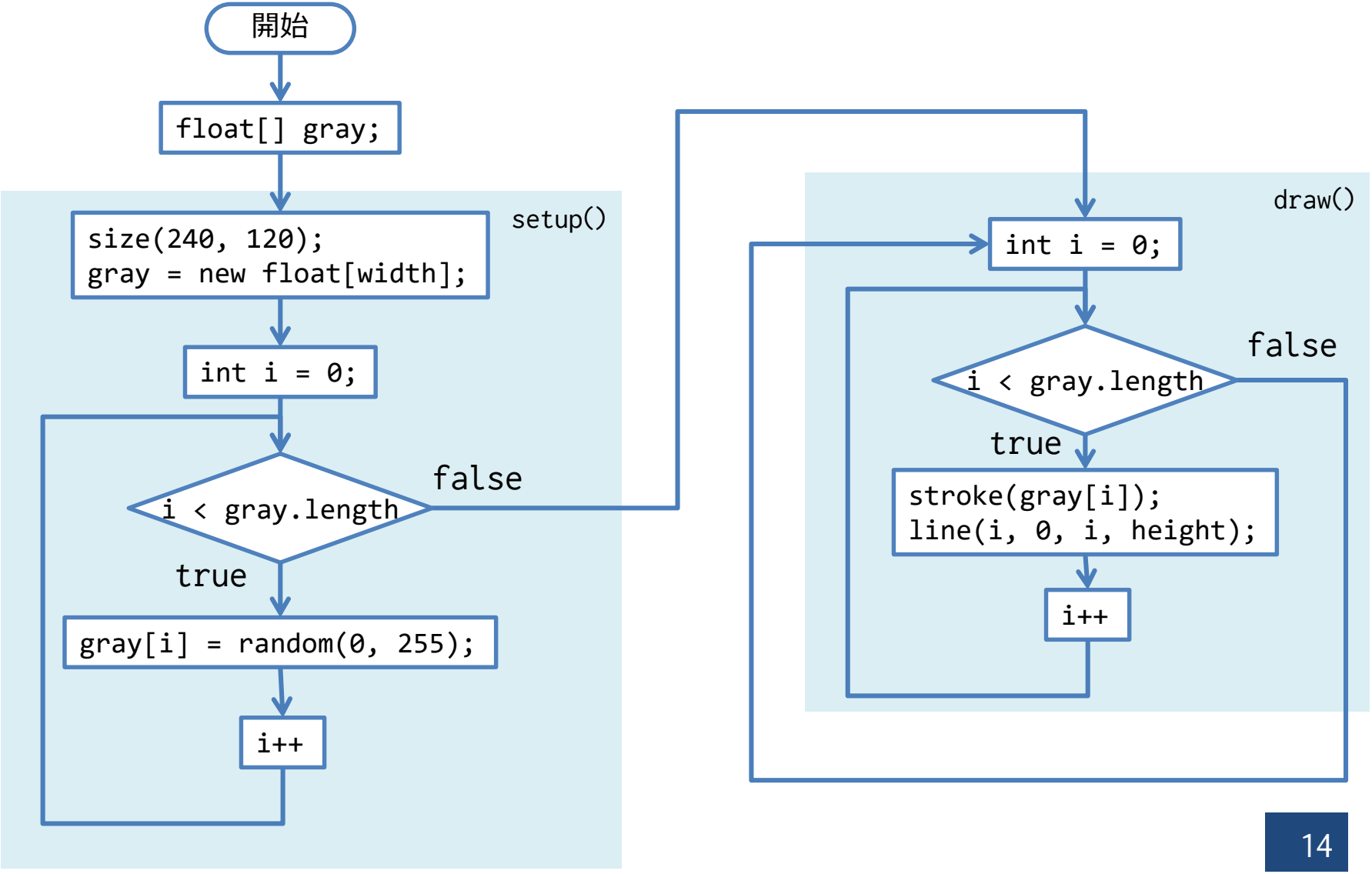
```
float[] gray;

void setup() {
  size(240, 120);
  gray = new float[width];
  for (int i = 0; i < gray.length; i++) {
    gray[i] = random(0, 255);
  }
}

void draw() {
  for (int i = 0; i < gray.length; i++) {
    stroke(gray[i]);
    line(i, 0, i, height);
  }
}
```

配列の長さ
配列名.length

forループを使って
配列の各要素に
乱数を使った初期値を代入

forループを使って
線の色を
配列の各要素の値で設定し
線を描く

配列と繰り返し


開始
float[] gray;
setup()
size(240, 120);
gray = new float[width];
int i = 0;
i < gray.length
false
true
gray[i] = random(0, 255);
i++
draw()
int i = 0;
i < gray.length
false
true
stroke(gray[i]);
line(i, 0, i, height);
i++

配列と繰り返し

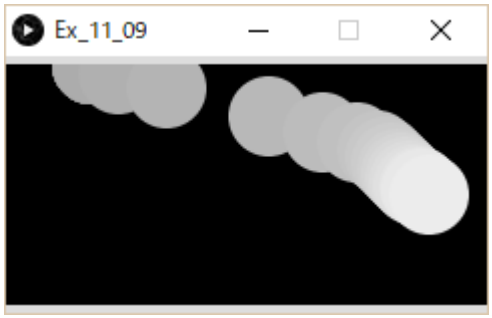
- Example 11-9 マウスを追跡する

```
int num = 60;
int[] x = new int[num];
int[] y = new int[num];

void setup() {
  size(240, 120);
  noStroke();
}
```

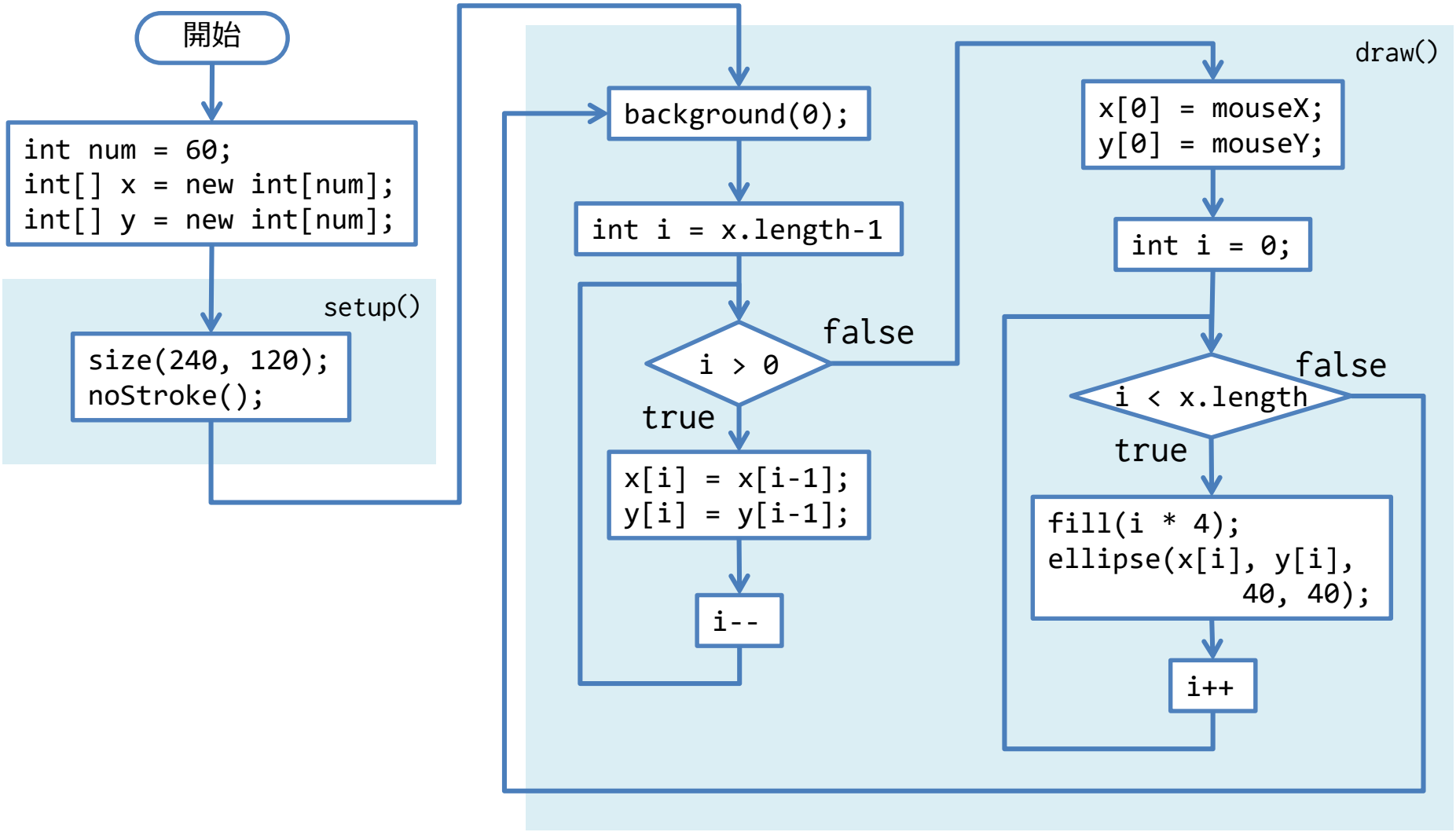
2つの配列x,yに過去60フレーム分の
マウスの座標データを記録し
円の動きを可視化

```
void draw() {
  background(0);
  for (int i = x.length-1; i > 0; i--) {
    x[i] = x[i-1];
    y[i] = y[i-1];
  }
  x[0] = mouseX;
  y[0] = mouseY;
  for (int i = 0; i < x.length; i++) {
    fill(i * 4);
    ellipse(x[i], y[i], 40, 40);
  }
}
```

配列内の値をシフト 教科書 P166



配列と繰り返し


開始
int num = 60;
int[] x = new int[num];
int[] y = new int[num];
setup()
size(240, 120);
noStroke();
draw()
background(0);
int i = x.length-1
i > 0
false
true
x[i] = x[i-1];
y[i] = y[i-1];
i--
x[0] = mouseX;
y[0] = mouseY;
int i = 0;
i < x.length
false
true
fill(i * 4);
ellipse(x[i], y[i],
40, 40);
i++

オブジェクトの配列

- オブジェクトの配列を宣言

  クラス名[] 配列名 = new クラス名[要素数];

```
JitterBug[] bugs = new JitterBug[33];
```

- setup()でオブジェクトの生成を繰り返し実行

```
for (int i = 0; i < bugs.length; i++) {
  float x = random(width);
  float y = random(height);
  int r = i + 2;
  bugs[i] = new JitterBug(x, y, r);
}
```

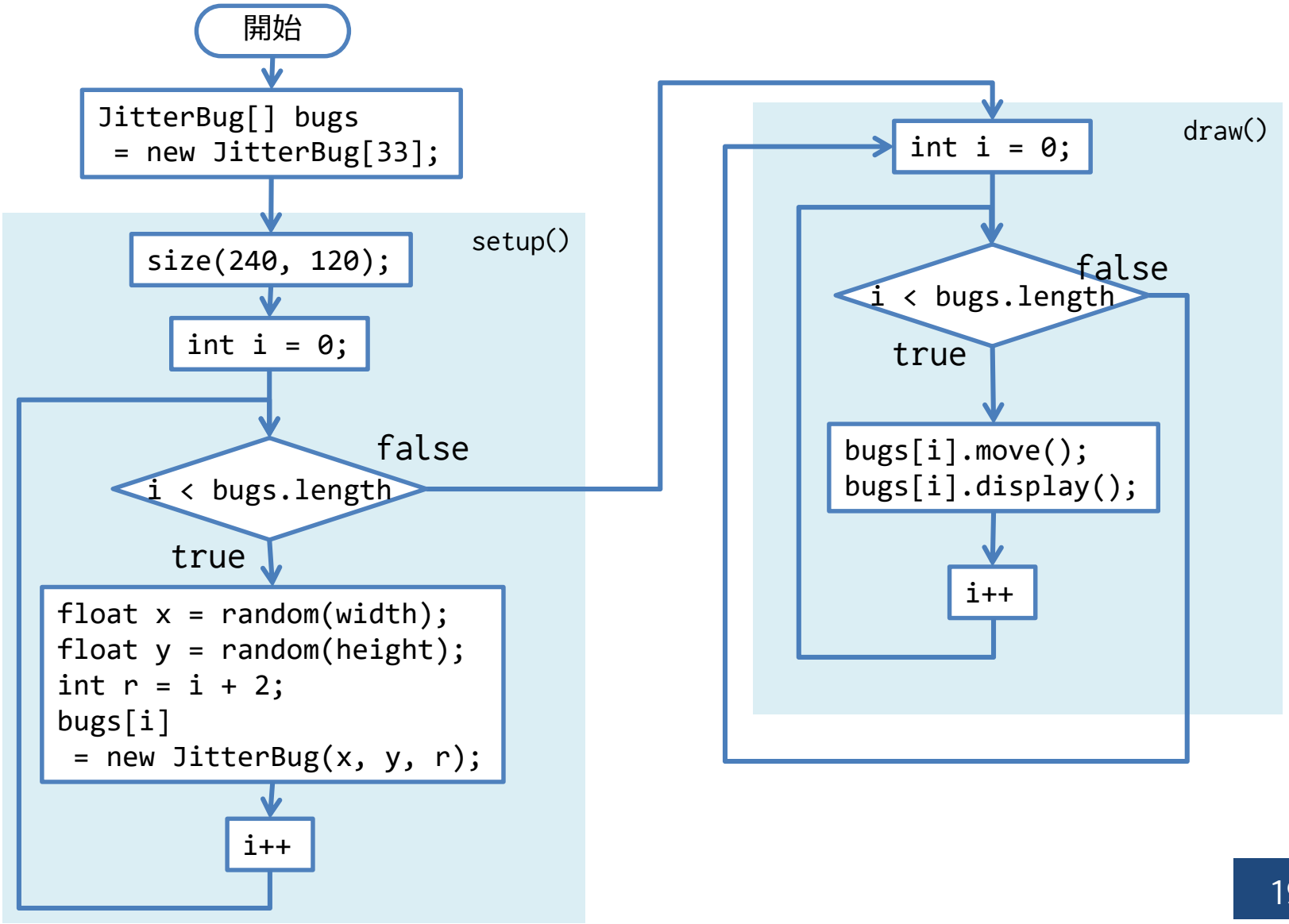
オブジェクトの配列

- Example 11-10 多数のオブジェクトを管理

```
JitterBug[] bugs = new JitterBug[33];

void setup() {
  size(240, 120);
  for (int i = 0; i < bugs.length; i++) {
    float x = random(width);
    float y = random(height);
    int r = i + 2;
    bugs[i] = new JitterBug(x, y, r);
  }
}

void draw() {
  for (int i = 0; i < bugs.length; i++) {
    bugs[i].move();
    bugs[i].display();
  }
}
```

```
class JitterBug {

  float x;
  float y;
  int diameter;
  float speed = 2.5;

  JitterBug(float tempX, float tempY,
                    int tempDiameter) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display() {
    ellipse(x, y, diameter, diameter);
  }
}
```

配列と繰り返し


開始
JitterBug[] bugs
= new JitterBug[33];
setup()
size(240, 120);
int i = 0;
i < bugs.length
false
true
float x = random(width);
float y = random(height);
int r = i + 2;
bugs[i]
= new JitterBug(x, y, r);
i++
draw()
int i = 0;
i < bugs.length
false
true
bugs[i].move();
bugs[i].display();
i++

オブジェクトの配列

- Example 11-11 新しいオブジェクト管理の方法

```
JitterBug[] bugs = new JitterBug[33];

void setup() {
  size(240, 120);
  for (int i = 0; i < bugs.length; i++) {
    float x = random(width);
    float y = random(height);
    int r = i + 2;
    bugs[i] = new JitterBug(x, y, r);
  }
}

void draw() {
  for (JitterBug b : bugs) {
    b.move();
    b.display();
  }
}
```

拡張forループ
教科書 P168

```
for(データ型 変数名：配列名){
  文
}
```

```
class JitterBug {

  float x;
  float y;
  int diameter;
  float speed = 2.5;

  JitterBug(float tempX, float tempY,
                    int tempDiameter) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display() {
    ellipse(x, y, diameter, diameter);
  }
}
```

オブジェクトの配列

- Example 11-12 画像のシーケンス

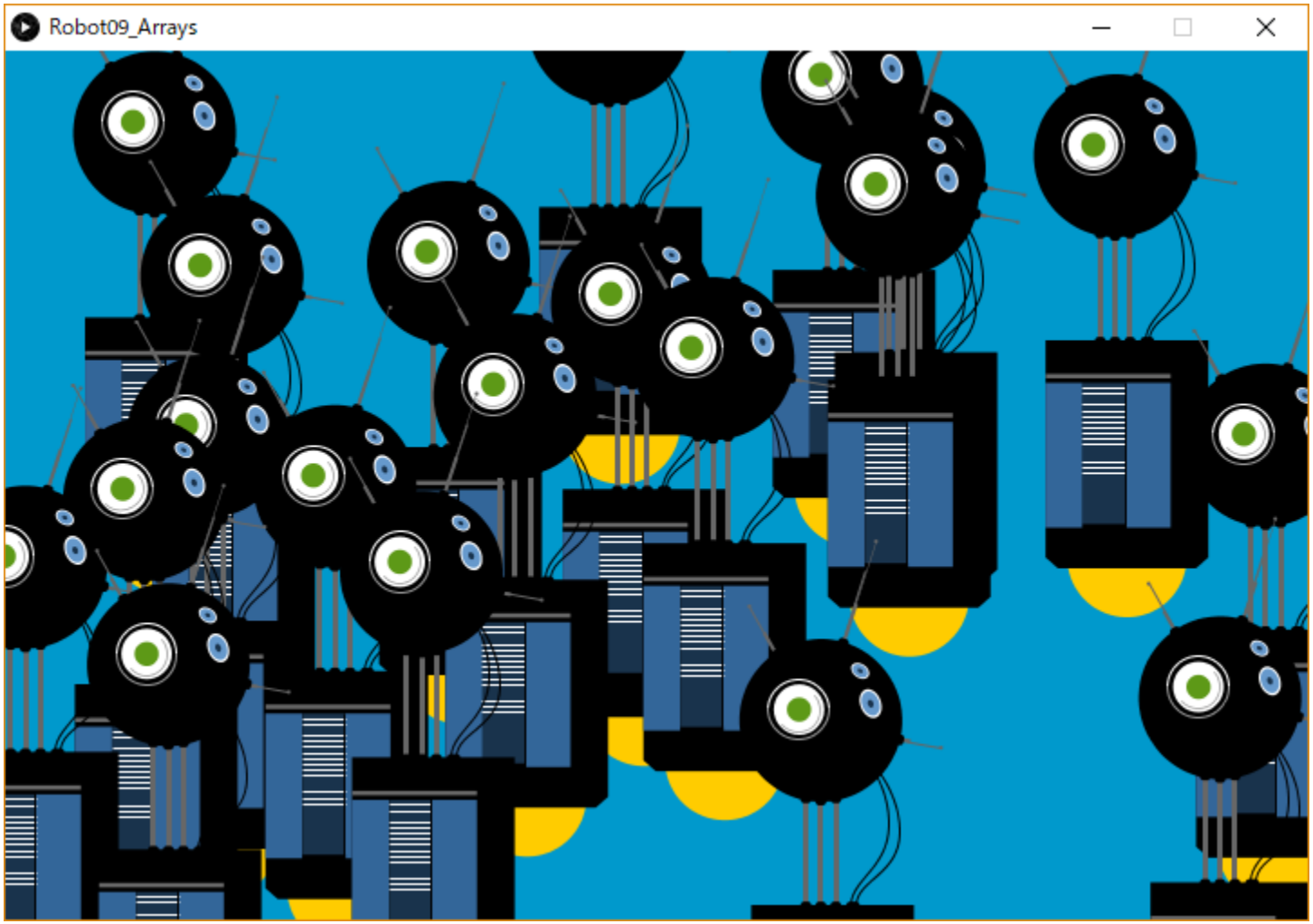
```
int numFrames = 12; // フレーム数
PImage[] images = new PImage[numFrames]; // 画像データを保存する配列
int currentFrame = 0;

void setup() {
  size(240, 120);
  for (int i = 0; i < images.length; i++) {
    String imageName = "frame-" + nf(i, 4) + ".png";
    images[i] = loadImage(imageName); // 画像データを読み込む
  }
  frameRate(24);
}

void draw() {
  image(images[currentFrame], 0, 0);
  currentFrame++;
  if (currentFrame >= images.length) {
    currentFrame = 0; // Return to first frame
  }
}
```

Robot 9: Arrays

```
Robot[] bots;

void setup() {
  size(720, 480);
  PShape robotShape = loadShape("robot2.svg");
  bots = new Robot[20];
  for (int i = 0; i < bots.length; i++) {
    float x = random(-40, width-40);
    float y = map(i, 0, bots.length,
                        -100, height-200);
    bots[i] = new Robot(robotShape, x, y);
  }
}

void draw() {
  background(0, 153, 204);
  for (int i = 0; i < bots.length; i++) {
    bots[i].update();
    bots[i].display();
  }
}
```

```
class Robot {
  float xpos;
  float ypos;
  float angle;
  PShape botShape;
  float yoffset = 0.0;

  Robot(PShape shape, float tempX, float tempY) {
    botShape = shape;
    xpos = tempX;
    ypos = tempY;
    angle = random(0, TWO_PI);
  }

  void update() {
    angle += 0.05;
    yoffset = sin(angle) * 20;
  }

  void display() {
    shape(botShape, xpos, ypos + yoffset);
  }
}
```

Robot 9: Arrays



Kadai_11_01

- 配列の各要素の値をコンソールに表示するプログラムを作る
  - 整数型配列 test を以下のように初期化

    ```
    int[] test = {100, 95, 73, 60, 60, 59, 25};
    ```

  - forループを使って、i番目の配列要素の値をコンソールに表示する処理を0番目から最後まで繰り返す

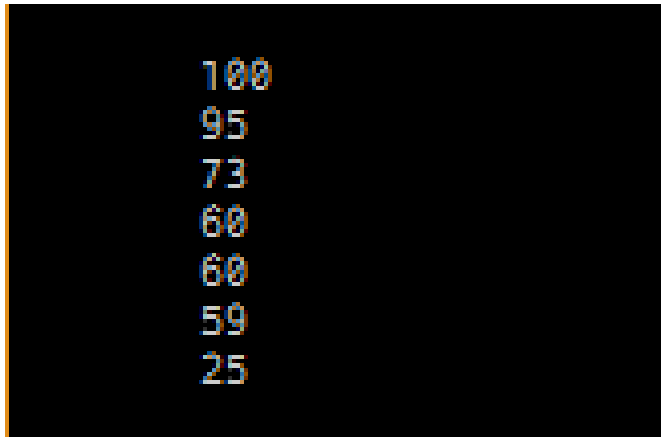
Kadai_11_01

配列testの宣言と初期化

```
void setup(){
  for(int i=0 ; i <
    println(
  }
}
```

配列testの長さ

i番目の配列要素



Kadai_11_02

- 配列を使って、7人のテスト点数の平均値を求めるプログラムを作る
  - 整数型配列 test を以下のように初期化

    ```
    int[] test = {100, 95, 73, 60, 60, 59, 25};
    ```

  - 平均値を入れる変数 float ave = 0;
  - 平均値を求める
    - 点数をすべて足し合わせ、人数で割る
  - 結果をコンソールに表示する

Kadai_11_02

配列testの宣言と初期化

```
float ave = 0;                    // 平均値 ave の宣言と初期化


void setup(){
  for(int i=0 ; i <
```

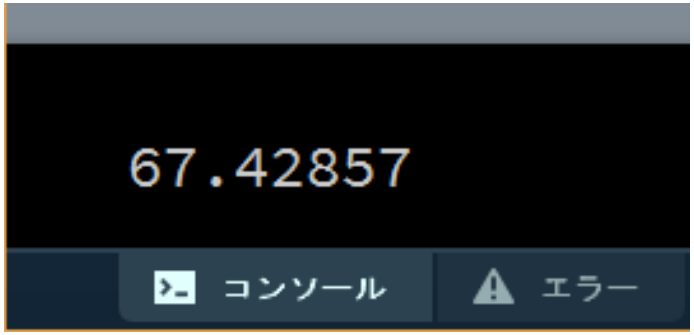
配列testの長さ

ave に i番目のテスト点数を加算する

```
  }
```

ave を人数で割る

```
  println(ave);
}
```



Kadai_11_03（任意提出）

- Kadai_10_01とExample 11-10を使って色付きバグを表示するプログラムを作る
  - Kadai_10_01で作成したJitterBugクラスを使う（解答例参照）
  - 処理内容はExample 11-10と同じ

Kadai_11_03

```
JitterBug[] bugs = new JitterBug[33];

void setup() {
  size(240, 120);
  for (int i= 0; i< bugs.l 配列bugsの長さ
    float x = random(width);
    float y = random(height);
    int r = i + 2;
    int g =        0以上255未満の整数の乱数を求める
    bugs[i] = new JitterBug(        ?, ?, ?, ?
  }
}

void draw() {
  for (int i= 0; i< bugs.l 配列bugsの長さ
    bugs[i].move();
    bugs[i].display();
  }
}
```

```
class JitterBug {

  float x;
  float y;
  int diameter;
  float speed = 2.5;
  int gray;

  JitterBug(float tempX, float tempY, int tempÐiameter,
int tempGray) {
    x = tempX;
    y = tempY;
    diameter = tempÐiameter;
    gray = tempGray;
  }

  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display() {
    fill(gray);
    ellipse(x, y, diameter, diameter);
  }

}
```